Benchmarking and Hybrid Modeling for Multilingual Public Service Chatbots under Noisy Conditions

Mannat Mehta
Department of CSE
Chandigarh University, Mohali, India

Yugam Rana
Department of CSE
Chandigarh University, Mohali, India

Prabjot Singh Bali
Department of CSE
Chandigarh University, Mohali, India

***Abstract*—Getting a government chatbot to work in India is more challenging than it seems. The main issue is not just technical; it is linguistic. A citizen trying to update their Aadhaar details might type something like "mera aadhaar update kaise kru" or "pan card ke liye apply karna hai bhai." Most NLP pipelines, which are primarily trained on clean English, struggle to process these inputs. These are not rare cases; this is how people actually communicate on their phones. In this paper, we benchmark four intent classification approaches for public service chatbots in these conditions. The four methods are: (i) TF-IDF-based SVM, (ii) BiLSTM, (iii) fine-tuned BERT, and (iv) a confidence-weighted hybrid that combines predictions from all three. We tested these on a domain-specific Aadhaar/PAN/Passport query dataset we created (around 5,000 samples, 15 classes, trilingual) and separately on CLINC150 (150 classes, 10 domains) as a larger test. We added synthetic noise mimicking real mobile typing patterns to 30% of the test data. What we discovered was somewhat surprising. Classical SVM performed better than BERT in noisy, low-resource conditions. This finding can be overlooked if one only runs benchmarks on clean data. The hybrid model, which uses the strengths of all three methods, performed best in both scenarios (F1 = 0.88 on CLINC150; noisy accuracy = 0.45 on the domain set). We also introduce a metric we call Accuracy Drop (Acc) to measure noise sensitivity and make it comparable across architectures. The paper concludes with practical advice for engineers dealing with real multilingual chatbot challenges.**

***Index Terms*—Government Chatbot, Multilingual NLP, Hinglish, BERT, BiLSTM, SVM, Hybrid Ensemble, Intent Classification, Noise Robustness, Code-Mixed NLP**

I. Introduction

India's digital public service infrastructure has expanded at a pace that few would have predicted even five years ago.DigiLocker, UMANG, Passport Seva Online—all of these services manage hundreds of millions of registered accounts, collectively. With that many users logging in comes a massive amount of queries, which agencies have increasingly turned to chatbots to manage by default—they're less expensive than call centers, quicker than email, and available 24/7.

The problem is that building a chatbot that actually works for Indian citizens is different from building one that scores highly on English language benchmarks. India has 22 scheduled languages. The reality on the ground is most smartphone users, especially among younger, urban, and semi-urban populations, speak in what linguists call Hinglish: a code-mixed language using words from both Hindi and English, written in the Latin alphabet, and rife with phonetic spellings that differ from speaker to speaker. You might see someone type "aadhaar mein address kaise change kare" or "passport renwal k liye kya documents chahiye." Neither of those queries would find a perfect match in any English language training dataset.

When chatbots fail on inputs like these, the cost to citizens is real. They revert to phone lines and in-person visits to get their issues resolved—creating the very backlog that digitization was supposed to solve. Repeated failures also corrode trust in the platform, which is a harder problem to fix than the technical one.

The NLP community has been working on this general area for a while. Classical SVM classifiers are fast and interpretable. BiLSTMs model sequential structure. BERT delivers strong contextual representations. But how each of these holds up specifically under multilingual, noisy, low-resource government chatbot conditions—and how they compare to one another in that setting—is less well understood than the headline benchmark numbers suggest. Hybrid combinations have been tried, but rarely benchmarked rigorously in this context.

This paper reports five contributions toward filling that gap:

1) **Comparative benchmarking** of SVM, BiLSTM, BERT, and a hybrid ensemble under both clean and noisy conditions, across two datasets that differ substantially in scale.
2) **A domain-specific dataset** of ≈5,000 trilingual (English/Hindi/Hinglish) public service queries spanning 15 intent classes, with four categories of injected synthetic noise.
3) **The Accuracy Drop metric** (ΔAcc), a simple and interpretable way to quantify how much a model's performance degrades under noise.
4) **A confidence-weighted hybrid ensemble** that outperforms all three individual baselines in both experimental settings.
5) **Deployment recommendations** grounded in the empirical results, intended to be usable by practitioners rather than just theoretically interesting.

Sections proceed as follows: related work (II), methodology (III), experimental setup (IV), results (V), comparative analysis

(VI), limitations (VII), and conclusion (VIII).

II. Related Work

A. Multilingual Government Chatbots

There is a small but growing body of work on chatbots for government services. Sangeetha et al. [1] used sentence embeddings for retrieval-based intent detection in a multilingual government chatbot, getting reasonable numbers on structured queries—but noisy or code-mixed inputs were not part of their evaluation at all. GovAssist [2] took an AI-driven approach to e-government support; the dataset details were sparse and the evaluation thin. PublicTalk [3] was an Indonesian-language chatbot with entity detection and feedback loops, though again with limited noise handling and no real benchmarking rigor. Zhang et al. [4] tried R-GCN with fuzzy logic for contextual reasoning in a multi-turn setting, but their evaluation was monolingual and noise-free. None of these papers, to our knowledge, attempt a systematic comparison of classical, deep learning, and transformer approaches under noisy multilingual conditions—which is the gap this work tries to address.

B. Transformer and Multilingual NLP

BERT [7] is the obvious starting point for any discussion of modern NLP intent classification. Its bidirectional pre-training allows the model to learn relationships in context missed by previous techniques, and fine-tuning for specific downstream tasks became common. RoBERTa [8] improved performance by modifying the pre-training recipe: bigger batches, more data, and removing next-sentence prediction XLNet [9] took a different approach, using permutation-based training to sidestep BERT's conditional independence assumption for masked tokens.

For Indian languages, MuRIL [16]—trained on 17 Indian languages plus their transliterations—is probably the most relevant multilingual pre-trained model, with documented strong performance on code-mixed Hinglish. We use the CLINC150 benchmark [14] (150 intents, 10 domains, 22,500 train samples) as our large-scale test bed.

C. Noise and Code-Mixing

BERT's WordPiece tokenization [11] gives it some built-in noise tolerance: a misspelled word gets decomposed into sub-word units, at least some of which may appear in the training vocabulary. But this only goes so far, especially when the misspelling is severe or the vocabulary is code-mixed.TF-IDF [13] methods deal relatively gracefully with light noise because statistics about term frequency partially endure corruption at the character level. Noise robustness was listed as an open problem alongside hybrid evaluation in a survey paper from 2022 [5] , and remains so today.

D. Gap Summary

Table I summarizes where prior work falls short and what we contribute.

TABLE I
Research Gaps and Contributions

Gap in Prior Work	This Paper's Contribution
No noisy/code-mixed evaluation for government chatbots	Systematic benchmark on noisy multilingual data
No cross-scale hybrid comparison	Classical + DL + transformer ensemble
Opaque datasets, limited reproducibility	Documented trilingual dataset with noise variants
No formal noise robustness metric	ΔAcc metric
Weak confidence aggregation strategies	Grid-searched confidence-weighted ensemble

III. Methodology

A. Overall Architecture

Rather than running queries through a single model and hoping for the best, our system sends each query simultaneously through three independent pipelines: SVM (TF-IDF features), BiLSTM (sequential embeddings), and BERT (contextual representations). Each produces a probability distribution over intent classes. These get combined via confidence-weighted voting.

The motivation is fairly intuitive. Different architectures have different failure modes. Under heavy noise, the model that usually performs best might degrade the most, while a simpler model maintains more stable predictions. By aggregating across architectures with weights proportional to their estimated confidence, the ensemble can compensate for individual model weaknesses dynamically. The pipeline itself has four stages: text preprocessing, feature extraction (parallel across the three models), independent inference, and weighted aggregation.

B. Datasets

1) Domain-Specific Dataset: We built a corpus targeting three areas of India's e-governance infrastructure: Aadhaar (biometric identity), PAN (tax registration), and Passport services. The dataset has approximately 5,000 queries across 15 intent classes. Intent classes include *status_check*, *document_update*, *appointment_booking*, *fee_inquiry*, *grievance_filing*, *eligibility_check*, *office_locator*, and several others. Class sizes range from 280 to 400 samples, keeping the distribution reasonably balanced.

Language distribution: 40% English, 30% Hindi (written in Latin script), 30% Hinglish. The Hinglish portion is the hardest to handle—it mixes Hindi vocabulary and grammar with English words, written phonetically, with no agreed-upon spelling conventions. The same intent can be expressed in completely different surface forms: "what is my Aadhaar status" (English), "mera aadhaar ka status batao" (Hindi), "aadhaar status check karna hai" (Hinglish)—all within the same class.

2) CLINC150: To check whether our findings held beyond this narrow domain, we also ran experiments on CLINC150 [14]: 22,500 training samples, 4,500 test samples, 150 intent classes across 10 domains including banking, travel, and home automation. This is a standard benchmark, which makes comparison with prior work straightforward.

3) Synthetic Noise: Noise was injected into 30% of test samples across four categories:

Orthographic Variance: character swaps and spelling errors mimicking mobile keyboard slips—"aply" for "apply," "passort" for "passport," "adhar" for "Aadhaar."

Phonetic Approximations: alternative romanizations of Hindi words, since there is no standard—"karwana" / "krwana," "zaroorat" / "jarurat."

Syntactic Informality: dropped articles, missing punctuation, telegram-style fragments—"pan card apply" in place of "I want to apply for a PAN card."

Lexical Shifts: informal synonyms and abbreviations that do not appear in formal text—"docs" for "documents," "govt" for "government."

Perturbations were applied singly and in combinations. The noise robustness metric is:

$$\Delta\text{Acc} = \text{Acc}_{\text{clean}} - \text{Acc}_{\text{noisy}} \tag{1}$$

A lower ΔAcc means the model degrades less under noise—i.e., it is more robust.

C. Preprocessing

All inputs are lowercased first—informal text is heavily mixed-case and case normalization removes a source of variance without losing much useful information. A regex filter strips characters that carry no semantic weight while preserving question marks (which can signal intent type).

For Hinglish tokens, we use English lemmatization on English-origin words and a phonetic normalization heuristic on Hindi-origin words in Latin script—mapping "zaroorat" and "jarurat" both to a canonical form, for instance. This cuts down on vocabulary fragmentation from phonetic variation.

After this shared preprocessing, the pipelines diverge. SVM uses TF-IDF with unigrams and bigrams, vocabulary capped at 10,000 tokens. BiLSTM uses integer-indexed token sequences padded to 40 tokens (this covers 95% of queries in our domain set). BERT uses its native WordPiece tokenizer. All neural models use 128-dimensional embeddings.

D. Individual Models

1) SVM: Linear-kernel SVM on TF-IDF features. TF-IDF weight:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log\left(\frac{N}{\text{df}(t)}\right) \tag{2}$$

This pushes domain-specific terms like "aadhaar," "passport," "apply" toward the top of the feature space while suppressing generic stop words. Margin maximization gives an overfitting-resistant boundary—important for high-variance Hinglish inputs. Balanced class weights handle the mild class imbalance.

2) BiLSTM: 128 hidden units per direction. Bidirectional processing is a good fit for code-mixed text, where the intent-bearing token might appear at the start (English convention) or the end (Hindi convention) of the sentence. Hidden state update:

$$h_t = \sigma(W_h x_t + U_h h_{t-1} + b_h) \tag{3}$$

Forward and backward states are concatenated, then fed through a dense layer with ReLU activation and softmax output. Dropout at 0.3. Adam optimizer, $lr = 10^{-3}$, up to 20 epochs with early stopping.

3) BERT: Fine-tuned `bert-base-uncased` with a single classification head appended to the `[CLS]` token. The self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{4}$$

captures long-range dependencies regardless of token distance—useful for informationally dense queries. We trained for 2–3 epochs with AdamW ($lr = 2 \times 10^{-5}$), batch size 16, max length 64. Short training is intentional: too many epochs and the model forgets its pre-trained representations.

E. Hybrid Aggregation

Each model $i \in \{\text{SVM}, \text{BiLSTM}, \text{BERT}\}$ outputs $P_i(y)$ over intent classes. Ensemble score:

$$\text{Score}(y) = \sum_{i=1}^{3} w_i \cdot P_i(y) \tag{5}$$

Final prediction:

$$y^* = \arg\max_y \ \text{Score}(y) \tag{6}$$

Weights ($w_{\text{BERT}} = 0.5$, $w_{\text{BiLSTM}} = 0.3$, $w_{\text{SVM}} = 0.2$) were found via grid search on the validation set, stepping in 0.1 increments with $\sum w_i = 1$, optimizing for F1 on noisy subsets. BERT gets the largest weight because it is generally the strongest model on clean inputs. SVM's non-zero weight (0.2) turns out to matter—it provides a lexical anchor when neural confidence collapses under noise.

IV. Experimental Setup

A. Implementation

Python 3.9 throughout. SVM: scikit-learn 1.2. BiLSTM and BERT: TensorFlow 2.11 and HuggingFace Transformers 4.26. All random seeds were fixed for shuffling, weight initialization, and dropout. Neural architectures used early stopping (patience = 3, monitoring validation loss). Table II gives the full hyperparameter breakdown.

B. Evaluation

Stratified 80:20 train-test split on both datasets. Domain-specific dataset: accuracy on clean and noisy splits, plus ΔAcc. CLINC150: macro-averaged precision, recall, F1. All numbers averaged over three independent runs with different random seeds.

TABLE II
HYPERPARAMETER CONFIGURATIONS

Model	Key Parameters	Optimizer
SVM	Linear kernel, C=1.0, balanced weights, top-10k TF-IDF unigrams + bigrams	L-BFGS
BiLSTM	128-d embed., 128 units/dir., dropout 0.3, seq. len. 40, batch 32	Adam, lr=10^{-3}
BERT	bert-base-uncased, len 64, batch 16	AdamW, lr=2×10^{-5}
Hybrid	w_B=0.5, w_L=0.3, w_S=0.2	N/A

V. RESULTS

A. Domain-Specific Dataset

Table III shows accuracy on clean and noise-injected versions of the Aadhaar-PAN-Passport dataset.

TABLE III
DOMAIN DATASET RESULTS (ΔACC: LOWER = MORE ROBUST)

Model	Clean	Noisy	ΔAcc	Rank
SVM	0.98	0.43	0.55	2nd
BiLSTM	0.98	0.14	0.84	4th
BERT	1.00	0.37	0.63	3rd
Hybrid (Ours)	0.99	0.45	0.54	1st

Clean-text accuracy is near-perfect across the board (0.98–1.00), which basically just confirms the implementations are correct. The interesting story is in the noisy column.

BiLSTM's collapse is striking: noisy accuracy drops to 0.14, giving a ΔAcc of 0.84. Once noise corrupts enough tokens to push sequences out of distribution, the LSTM gating mechanisms have nothing useful to work with—sequential dependency modeling breaks down when the token sequence itself is malformed.

BERT does better (noisy accuracy 0.37, ΔAcc = 0.63), but not as well as its clean-data performance might suggest. With only $\sim$5,000 training examples, the fine-tuned model does not see enough noisy variation to generalize robustly. WordPiece tokenization helps somewhat—corrupted words get split into sub-units, some of which may be in-vocabulary—but it is not a complete fix.

SVM is the surprise here. It achieves noisy accuracy 0.43 and ΔAcc = 0.55—better than BERT under these conditions. The reason is that TF-IDF features are partially robust to orthographic noise: corrupted tokens often share enough characters with their clean forms that they still contribute some signal to the term frequency counts.

The hybrid edges out all three (ΔAcc = 0.54, noisy accuracy = 0.45) by using SVM's lexical stability as a fallback when BERT's confidence degrades.

B. CLINC150

Table IV shows results on the large-scale benchmark.

TABLE IV
CLINC150 RESULTS (150 INTENT CLASSES)

Model	Prec.	Rec.	F1	Rank
SVM	0.81	0.79	0.80	3rd
BiLSTM	0.72	0.66	0.69	4th
BERT	0.90	0.85	0.87	2nd
Hybrid (Ours)	0.91	0.85	0.88	1st

At this scale ($\sim$22,500 training samples), BERT dominates the individual models (F1 = 0.87 vs. SVM's 0.80 and BiLSTM's 0.69). Given sufficient data, its contextual representations are clearly superior. The hybrid still comes out on top (F1 = 0.88), largely because SVM's conservative lexical matching cuts down on false positives in ambiguous intent categories.

VI. ANALYSIS AND DISCUSSION

A. The Crossover Effect

Table V puts both experimental settings side by side.

TABLE V
CROSS-DATASET SUMMARY

Model	Domain (Noisy Acc.)	CLINC150 (F1)
SVM	0.43	0.80
BiLSTM	0.14	0.69
BERT	0.37	0.87
Hybrid (Ours)	0.45	0.88

The SVM/BERT ranking inverts depending on which setting you look at. SVM beats BERT on the small noisy dataset; BERT beats SVM by a wide margin on CLINC150. This is a crossover effect, and it matters practically: if you only ever test on clean, large-scale benchmarks, you will conclude that transformers are always the right choice. They are not—not when your data is scarce and your users type messily.

B. Why BiLSTM Underperforms Consistently

BiLSTM is never the best option in either setting. On clean CLINC150, it trails BERT by 18 F1 points. On noisy domain data, it nearly collapses. This points to a structural weakness: recurrent architectures are more vulnerable to token-level corruption than either TF-IDF (which is bag-of-words and therefore order-insensitive) or BERT (which at least has sub-word tokenization as a backstop). Without the benefit of large-scale pre-training, BiLSTM cannot develop the robust representations that BERT acquires. It ends up being more expensive than SVM without being more robust than BERT—an awkward position.

C. Why the Hybrid Works

The three models fail differently. BiLSTM loses track of intent when the key token appears at the end of a long sentence (as is common in Hindi-influenced syntax)—by the time the LSTM reaches "aadhaar" in "main ye bata chahta hun ki mera aadhaar," the hidden state has been diluted by preceding filler. BERT stumbles when heavy noise produces sub-word

fragments that do not map to any coherent representation—"pssport" might decompose into tokens that have no semantic relationship to "passport" in the fine-tuned embedding space. SVM fails when the query requires understanding negation or syntax beyond bigram co-occurrence—"I do not want to update my address" would likely be mis-predicted if "update address" is a strong bigram signal.

The confidence-weighted aggregation handles this naturally. When BERT's softmax output becomes flat (low peak probability), the SVM's sparse-but-reliable bigram signals—"apply," "status," "correction," "aadhaar"—take over the vote. The result is a system that is more consistently right than any single model.

D. Deployment Guidance

Four practical recommendations from these results:

1) **Small agencies, limited data, noisy users**: start with SVM. It is cheap, fast, interpretable, and holds up better than BERT in this regime.
2) **Large national platforms** (UMANG, DigiLocker): fine-tune BERT or a multilingual variant (mBERT, MuRIL). The data volume is likely sufficient to make it worth it.
3) **High-stakes services** (Aadhaar, PAN): deploy the hybrid with a confidence threshold—we suggest routing to a human operator when $P_{\max} < 0.6$.
4) **Edge/rural deployments**: SVM offers the smallest memory footprint. If accuracy requirements are higher, consider distilling the hybrid into a compact student model.

VII. Limitations

A few things we are aware of that we could not fully address here.

The domain dataset is small and entirely synthetic in its noise. Real production query logs from government portals would give a much better picture of what noise distributions actually look like in the wild—our four noise categories are reasonable approximations, but they are still approximations. Getting access to anonymized logs from DigiLocker or UMANG would be the right next step empirically.

The hybrid weights are static. They were tuned on a validation set and then fixed. In reality, noise patterns shift—mobile keyboard autocorrect changes, regional usage evolves, new document types get added. Adaptive weight updates based on live confidence signals would make the system more robust over time.

We only evaluated `bert-base-uncased`. The obvious alternative for this setting is MuRIL [16], which was explicitly pre-trained on Indian languages and their transliterations. We expect it would substantially outperform `bert-base-uncased` on Hinglish, and future work should test this systematically.

Finally, BERT and the hybrid incur latency in the 50–200 ms range on commodity hardware. For a real-time chatbot, this may be borderline. Model compression—quantization, knowledge distillation into a smaller architecture—would be needed before deploying at scale.

VIII. Conclusion

A. What We Found

We set out to answer a practical question: which NLP architecture should you use for intent classification in a multilingual Indian government chatbot, especially when user inputs are noisy? The answer is not simple, and that is itself the main finding.

In low-resource, high-noise conditions, classical SVM outperforms fine-tuned BERT. In large-scale, cleaner conditions, BERT dominates. The hybrid ensemble is consistently the safest choice—it is never far behind the best single model, and it outperforms all of them in both settings we tested. The ΔAcc metric we introduced gives a clean, comparable way to quantify noise robustness, which is not something prior benchmarks have offered.

The deeper point is that "just use BERT" is not always the right call. The relationship between model complexity and robustness is nonlinear, and dataset size is the key moderating variable. Practitioners designing public service chatbots need to calibrate their architecture choice to the data they actually have—not the data they wish they had.

For high-stakes services like Aadhaar and PAN, where a misclassified intent might mean a citizen is denied access to something they urgently need, the hybrid with confidence-thresholded human escalation is the approach we would recommend.

B. Next Steps

The most pressing extensions are: (1) evaluating MuRIL and IndicBERT for Hinglish specifically; (2) implementing adaptive ensemble weights that respond to real-time noise signals; (3) extending preprocessing to Devanagari and other regional scripts; (4) testing with real production query logs rather than synthetic noise; and (5) distilling the hybrid into a model lightweight enough for rural kiosk deployment.

References

[1] B. Sangeetha et al., "Multilingual Chatbot for Government Services," in *Proc. IEEE Int. Conf. Data Science and Business Systems (ICDSBS)*, 2025.

[2] A. Alsubayhay and M. Abdalla, "GovAssist: AI-driven E-Government Chatbot," in *Proc. Int. Conf. Artificial Intelligence Applications*, 2023.

[3] M. E. Fauzi et al., "PublicTalk: Indonesian NLP-Based Government Chatbot," *Jurnal Sains Student Research*, 2023.

[4] Y. Zhang et al., "Multi-turn Government Service Chatbot using R-GCN and Fuzzy Logic," in *Proc. IEEE Int. Conf. Intelligent Systems*, 2022.

[5] F. Liverier, "A Literature Survey of Recent Advances in Chatbots," *Information*, vol. 13, no. 4, 2022.

[6] A. Sharma et al., "Advanced AI Techniques for Personalized Customer Support Using Intelligent Chatbots," *J. Artificial Intelligence Applications*, 2025.

[7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[8] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.

[9] Z. Yang et al., "XLNet: Generalized Autoregressive Pretraining for Language Understanding," in *Proc. NeurIPS*, 2019.

[10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, 2017.

[12] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation," in *Proc. EMNLP*, 2014.
[13] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge Univ. Press, 2008.
[14] C. Xu et al., "CLINC150: A Benchmark Dataset for Intent Classification," *arXiv preprint arXiv:2003.04807*, 2020.
[15] D. Nozza, F. Bianchi, and D. Hovy, "Measuring Harmful Sentence Completion in Language Models," in *Proc. ACL*, 2023.
[16] S. Gumma et al., "Multilingual NLP for Low-Resource Languages: Challenges and Opportunities," *IEEE Access*, 2024.